

Recipes And Allergens

Alessandra d'Apolito, Cristina Papi, Marco Rosato

Abstract

This project aims to develop a data integration system to have not only a database of recipes, but also the corresponding allergens. Therefore, the recipes from TheMealDB API are combined with allergen information from Kaggle. The two datasets were collected and processed using MongoDB Atlas with a normalized star schema. The system implements a dual integration approach using both lexical and semantic joins to map ingredients to potential allergens. To implement this dual strategy, lexical joins were used for exact string matching, while semantic joins leveraged Sentence Transformer models (all-mpnet-base-v2) to perform AI-driven similarity analysis. The semantic approach captures implicit allergen relationships that traditional matching methods miss. This demonstrates how AI-enhanced data integration can bridge gaps in traditional matching methods, providing a robust solution for food safety applications that supports individuals with food allergies in making safer dietary decisions.

Contents

1	Introduction	1
2	Data Acquisition	2
2.1	API Data Collection	2
2.2	Kaggle Dataset Download . . .	2
3	Data Preparation	2
3.1	Data Preparation Recipes Dataset	2
3.2	Data Preparation Allergies Dataset	3
3.3	Data Transformation Process .	3
4	Allergen Lexical Mapping	3
4.1	Data Storage	3
4.2	Data Model	3
5	Allergen semantic detection	5
6	Conclusions	6

making access to clear and reliable allergen information more important than ever. The goal of our project is to provide a comprehensive and accessible resource that helps individuals with food allergies identify potential risks associated with recipes and ingredients.

We have developed a unified recipe database integrated with allergen information, combining data from two complementary sources: TheMealDB API and a Kaggle allergen dataset. Through careful data cleaning, we aligned and integrated the two datasets using both lexical and semantic matching techniques.

It has been implemented a MongoDB-based architecture, enables efficient querying and provides users with enriched recipe data that clearly highlights potential allergenic ingredients.

To carry out this project, we used Python, which allowed us to design and implement the entire data processing.

1 Introduction

In recent years, food allergies have become an increasingly relevant public health concern,

2 Data Acquisition

We collect data from two different sources, TheMealDB API and "Food Allergens and Allergies" from Kaggle

We collect data from different sources to have a completed Recipe dataset, containing detailed information about various meals, their ingredients, and their allergies. It helps us to make a richer analysis.

2.1 API Data Collection

A dataset was collected from TheMealDB API^[2], an open source database that provides information on a wide variety of meals and recipes. Using a Python script, requests were made for recipes starting with each letter of the alphabet. The API responses were parsed in JSON format, and relevant data entries

were extracted, processed, and stored for further analysis. It has 302 observation and 54 variables.

2.2 Kaggle Dataset Download

In addition to the API, a structured dataset was downloaded from Kaggle, a well-known platform for data science competitions and datasets. This dataset complements the API data by providing more detailed or curated information, which include additional attributes on the food allergies^[1]. It has 184 rows and 5 columns.

3 Data Preparation

After we collect our data from API and Kaggle, we clean our data to analyze them

3.1 Data Preparation Recipes Dataset

For the first dataset, we loaded the API data directly into a DataFrame to simplify the data cleaning process. We counted the missing values and removed all variables that contained only None values. In particular, four variables, `strMealAlternate`, `strImageSource`, `strCreativeCommonsConfirmed`, `dateModified`—, had 302 missing values and were removed.

The variables related to ingredients and measures contained a large number of missing values, but we decided to retain them to avoid losing important information about ingredients, which might be linked to unknown allergies. We also removed the `starts_with`

variable, which was only used to facilitate the data acquisition process.

Next, we normalized the ingredient and measurement string values by removing leading and trailing whitespace and converting all text to lowercase, in order to make them easier to compare and analyze.

Finally, we converted most of the measurements into grams and milliliters to ensure a more consistent and understandable dataset (since many of the original units were uncommon in Italy), thus facilitating the analysis. After that, we removed the 20 columns containing the original measurement values.

3.2 Data Preparation Allergies Dataset

For the second dataset, we also checked for missing data. There were only 22 missing values in the **Allergy** variable, possibly related to unknown allergies, potentially caused by specific ingredients. Therefore, we decided to fill these missing entries with the label "Unknown".

As in the first dataset, we normalized the ingredient and measurement string values by trimming whitespace and converting text to

lowercase.

3.3 Data Transformation Process

The transformation from wide-format to normalized structure involved unpivoting ingredient columns (`strIngredient1`, `strIngredient2`, etc.) and their corresponding measures (`Measure1_converted`, `Measure2_converted`, etc.) into a long-format structure. This process created a clean many-to-many relationship between recipes and ingredients while preserving measurement information.

4 Allergen Lexical Mapping

Join through MongoDB

4.1 Data Storage

The system uses MongoDB Atlas, a cloud-hosted database platform, chosen for several reasons:

1. **Operational Simplicity:** MongoDB Atlas eliminates infrastructure management and the web-based management interface enables real-time collection inspection and job verification, streamlining the development process.
2. **Global Accessibility:** The cloud-hosted environment ensures universal data access for authorized users, facilitating collaborative development and deployment across distributed teams.
3. **Schema Flexibility:** MongoDB's document-based architecture adapts seamlessly to evolving application requirements. The schema-less design allows structural modifications without costly processes.
4. **Developer Experience:** The JSON-like document format aligns naturally with modern programming languages

and APIs, reducing impedance mismatch and simplifying data manipulation operations.

The implementation leverages MongoDB's free tier, which proved sufficient for our dataset size. Additionally, the data model was optimized to minimize storage requirements while maintaining query performance.

4.2 Data Model

Star Schema Implementation

The data architecture follows a star schema pattern to optimize storage efficiency and query performance. The original flat recipe dataset was decomposed into multiple normalized collections:

Core Collections

- **Recipes (Fact Table):** Contains comprehensive recipe information including cooking instructions, categories etc.
- **Ingredients (Dimension Table):** Stores ingredient-measure pairs for each recipe, identified by `idMeal`.

- **Allergies (Dimension Table):** Maintains food allergy information and allergen classifications.

Aggregated Collections

- **recipes_to_ingredients:** Pre-computed aggregations for complex recipe-ingredient relationship queries.
- **ingredients_with_allergies:** Optimized views for ingredient-allergy relationship analysis.
- **recipes_with_allergies:** Comprehensive recipe-allergy relationship mappings.

Indexing Strategy

To ensure optimal query performance, a comprehensive indexing strategy was implemented:

Index Implementation

- **Recipes:** Single-field index on `idMeal` (primary key)
- **Ingredients:** Compound index on (`idMeal`, `ingredient`, `measure`) to ensure uniqueness and optimize lookup operations
- **Allergies:** Single-field index on `Food` column for efficient allergen queries

Aggregation Pipelines

Pre-computed aggregation pipelines were developed and stored as MongoDB collections to handle complex multi-table queries efficiently. These pipelines establish relationships between:

- **Ingredients - Allergies:** Mapping ingredients to potential allergens
- **Recipes - Ingredients:** Linking recipes to their ingredient compositions
- **Recipes - Allergies:** Providing direct recipe-to-allergen relationships for rapid

filtering

This approach significantly reduces query complexity and improves response times for common use cases such as allergen-free recipe searches and nutritional analysis.

Data Quality Assurance:

An analysis of the star schema model was conducted to assess the dataset's structure, completeness, and consistency, with particular emphasis on ingredient and allergen information.

The dataset comprises 302 recipes, 3,243 ingredients, and 183 allergen types, organized across fact and dimension tables. While the overall structure is well-organized, several issues affecting data quality were identified.

In terms of data consistency, all ingredients were appropriately linked to corresponding recipes, and vice versa, indicating that the table joins functioned correctly.

A high frequency of certain ingredients across multiple records was observed; for instance, "butter" appears in 117 entries. This repetition does not indicate duplication errors but rather reflects the valid reuse of common ingredients across different recipes. As such, this pattern aligns with the intended design of the data model.

Further validation highlighted potential data quality concerns, including recipes containing fewer than three ingredients and ingredients with vague or non-standard measurement descriptions (e.g., entries labeled as "handful" or "dash"). These entries were left unmodified due to the ambiguity and lack of precision in the original data, which made them unsuitable for standardization.

With regard to allergen mapping, only partial coverage was achieved using lexical matching techniques. However, semantic matching allowed for complete (100

5 Allergen semantic detection

via MPNet

The previous method, while straightforward, quickly revealed its limitations, primarily its inability to capture relationships between semantically similar but lexically different terms. For example, a recipe listing “mozzarella” would not be linked to the “Milk” allergen if the reference database only contained broader terms like “cheese.” To address this gap, we turned to semantic mapping. Specifically, we used a Sentence Transformer model, MPNet^[4], which proved far more effective than traditional static embeddings like Word2Vec^[3].^{*} There were two key reasons behind this choice:

- **Understanding of Phrases and Context:** Unlike Word2Vec, which generates embeddings at the word level, MPNet creates a unified representation for entire phrases. This allows it to capture the meaning of multi-word expressions like “extra virgin olive oil” in a single vector, which is crucial in culinary contexts where meaning often emerges from word combinations.
- **Robustness to Domain-Specific Terms:** The language of food is rich with specialized terminology. Sentence Transformers use sub-word tokenization, enabling them to effectively handle rare or domain-specific words like “pancetta” or “guanciale,” which might be out-of-vocabulary for tradi-

tional models.

Our approach involves a two-step process. First, we use the MPNet model to embed our entire vocabulary of unique recipe ingredients and all food items from our allergen reference database into a shared, high-dimensional vector space. Within this space, we use cosine similarity to find the most conceptually similar food item for each ingredient. If the similarity score for this best match exceeds a predefined threshold of 0.65, we consider it a confident semantic link. Second, once an ingredient is successfully matched to a food item (e.g., “mozzarella” to “cheese”), we use this link to retrieve the corresponding allergen from our reference database (e.g., “cheese” maps to “Milk”). By leveraging the strengths of MPNet in this way, we moved beyond rigid string comparisons to build a system that better understands the “language of food,” resulting in more accurate and comprehensive allergen detection across our dataset.

However, despite its effectiveness, this AI-based approach should not be considered the sole or primary method for allergen detection. Due to the computational cost of evaluating all $n \times m$ pairwise combinations between ingredients and allergens, semantic mapping is best used as a complementary tool: one that enhances, rather than replaces, more efficient rule-based or domain-specific matching strategies.

^{*}We adopted the `sentence-transformers/all-mpnet-base-v2` model, available at <https://huggingface.co/sentence-transformers/all-mpnet-base-v2>.

6 Conclusions

As a result of this work, we have developed a comprehensive recipe database enriched with allergen information, offering a valuable resource for individuals with food allergies by helping them identify potential risks more effectively. This project demonstrates how AI-driven approaches can be very helpful in real-world assumptions. The lexical approach provides accurate results only in cases of perfect match between the ingredients. The semantic approach using Sentence Transformers identified additional allergens that would have been missed by traditional string matching alone, significantly improving the system’s coverage and reliability. Furthermore it has been implemented a scalable MongoDB-based architecture that handles complex many-to-many relationships between recipes, ingredients, and allergens. The normalized star schema design enables efficient querying while maintaining data integrity through strategic indexing.

Future improvements could include:

1. Expanding the allergen database with more comprehensive food-allergy mappings
2. Implementing machine learning models trained specifically on food and allergen data
3. Adding user feedback mechanisms to continuously improve allergen detection accuracy
4. Extending the system to include severity levels for different allergic reactions

References

- [1] Food allergens and allergies. <https://www.kaggle.com/datasets/boltcutters/food-allergens-and-allergies>.
- [2] Recipes. <https://www.themealdb.com>.
- [3] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg Corrado, and Jeffrey Dean. Distributed representations of words and phrases and their compositionality. *Advances in Neural Information Processing Systems*, 26, 2013.
- [4] Kaitao Song, Xu Tan, Tao Qin, Jianfeng Lu, and Tie-Yan Liu. MpNet: Masked and permuted pre-training for language understanding. In *Advances in Neural Information Processing Systems*, volume 33, pages 16857–16867, 2020.